



Parallel Programming using the Message Passing Interface (MPI)

张超

中国人民大学信息学院

Slides mainly from Prof Stephen Neethling at Imperial College London

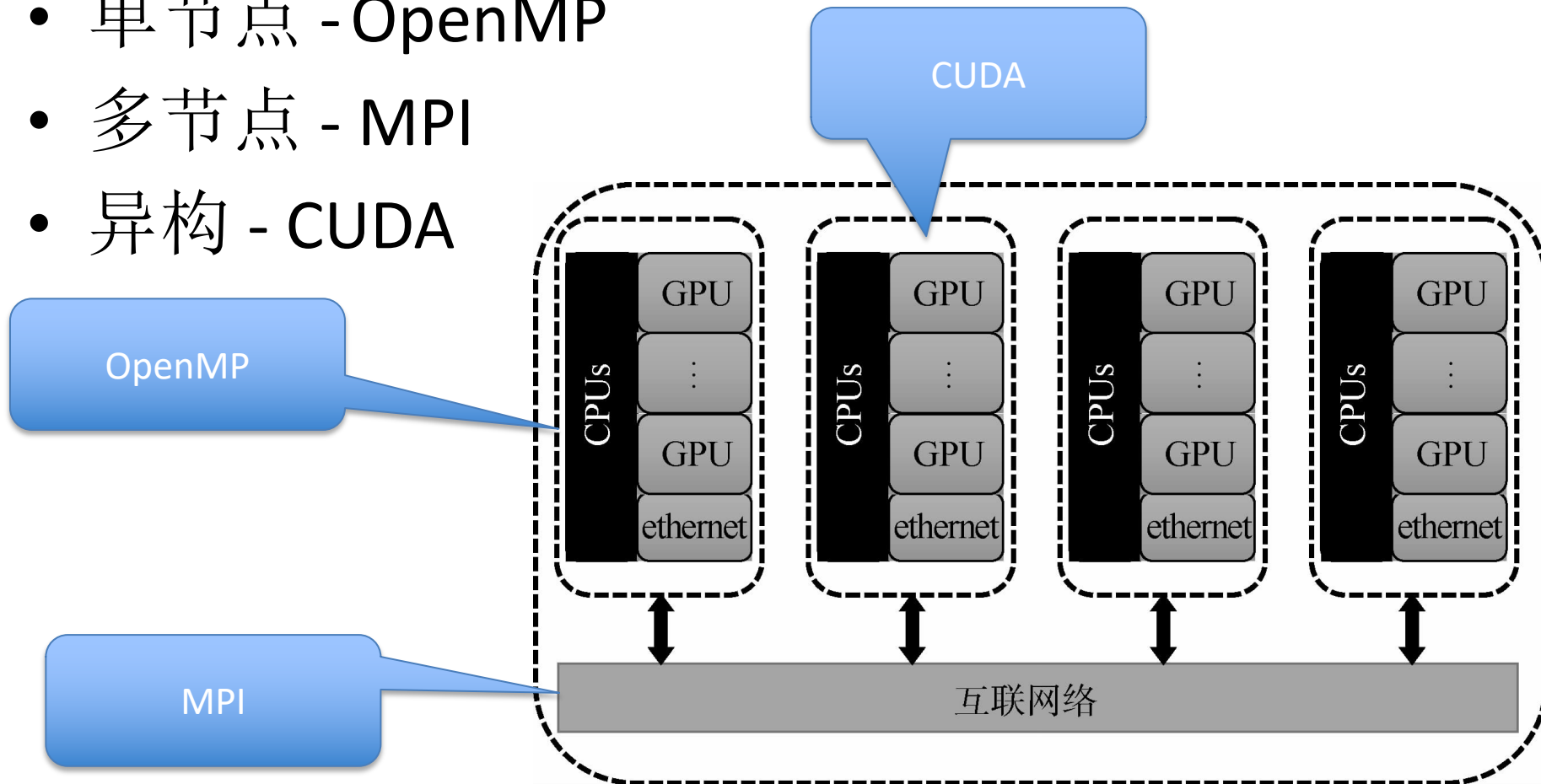


目录

- Introduction
- Communication
- Problem Parallelism

课程体系

- 单节点 - OpenMP
- 多节点 - MPI
- 异构 - CUDA





A very basic MPI program

```
#include <mpi.h>
#include <iostream>
```

```
using namespace std;
```

```
int id, p;
```

```
int main(int argc, char *argv[])
{
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &p);
```

```
    cout << "Processor " << id << " of " << p << endl;
    cout.flush();
```

```
    MPI_Finalize();
}
```

Header for all the MPI functions

main needs to take in arguments as they must be passed on to MPI_Init

Needs to be called to setup MPI communications

Reads the rank (number) of the current process

Reads the total number of processes that have been assigned

We will discuss later the idiosyncrasies of writing to stdout under MPI

Allows MPI to exit gracefully

Without this some communications on processes that have not yet finished running might not complete resulting in errors



An Important Note

- You need to think very differently about programs implemented with OpenMP compared to MPI
- In OpenMP you start a single copy of your program
 - Within that single program multiple threads will be spawned (and potentially go out of existence), but you have always got recourse to serial execution of certain sections of the code
- In MPI you start multiple copies of your program
 - The only way that one copy of your program (i.e. one process) knows what another copy is doing is through a communication.
 - There is no possibility of recourse to a serial section of code that knows everything
 - All aspects of the code are thus inherently occurring in parallel



MPI_COMM_WORLD

- In MPI all communications require a communicator, which is essentially a list of processes involved in the communication
- **MPI_COMM_WORLD** is a pre-defined communicator that includes all the processes
 - In this course we will only be having all the cores do the same thing (except for possibly the zero process) and therefore will only use MPI_COMM_WORLD
- It is possible to set your own up
 - Used if you have, for instance, different processes doing different tasks



MPI_Barrier

- Because there is no guarantees about the order in which data sent to stdout by different processes is written out, it would be useful for debugging purposes that a message, for instance, written out when every process has passed a certain point
- **MPI_Barrier** allows you to do that as it requires every process to get to that point before it continues
 - It can thus be useful to call MPI_Barrier and then write an output on only a single process (usually zero) as you can be guaranteed that every process has got to that point
 - This is best done for debugging purposes – Having MPI_Barriers in your code can make it inefficient (processes unnecessarily waiting for other processes)



An example with MPI_Barrier

```
#include <mpi.h>
#include <iostream>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    cout << "Processor " << id << " of " << p << endl;
    cout.flush();

    MPI_Barrier(MPI_COMM_WORLD);
    if (id == 0) cout << "Every process has got to this point now!" << endl;

    MPI_Finalize();
}
```



目录

- Introduction
- **Communication**
- Problem Parallelism



Types of Communication

- In MPI there are two main types of communication

Point-to-point communication:

- A communication involving a pair of processes
- Good if a process only needs to communicate with a limited number of other processes

Collective communication:

- A communication involving all the processes
 - Inefficient – Sending data from one process to every other process involves $p-1$ communications from the originating node
 - Collective communications can do things like relaying data



Blocking point-to-point communication

- We will initially do blocking point-to-point communication
 - Easier to code and shows the basic idea
 - Later on we will look at non-blocking communications
- Two main functions involved:
 - `MPI_Send` – Sends data to another process
 - `MPI_Recv` – Receives data from another process
- Because these are blocking, you must be careful of order
 - If a process tries to send data without all the previous receives being handled, there will be a block in communication and the code will hang or crash
 - We will later look at using probe as a way around this problem



MPI_Send

```
int MPI_Send(  
    void* data,  
    int count,  
    MPI_Datatype datatype,  
    int destination,  
    int tag,  
    MPI_Comm communicator)
```

- **data** is a pointer to the data to be sent
- **count** is the number of items to be sent
number generator
- **datatype** is the MPI data type to be sent
- **destination** is the id of the process that is
to receive the data
- **tag** is an identifier for the communication
- For both MPI_Send and MPI_Recv the
return value indicates error or success



MPI_Recv

```
int MPI_Recv(  
    void* data,  
    int count,  
    MPI_Datatype datatype,  
    int source,  
    int tag,  
    MPI_Comm communicator,  
    MPI_Status* status)
```

- **data** is a pointer to the data to be sent
- **count** is the number of items to be sent
number generator (maximum number)
- **datatype** is the MPI data type to be sent
- **source** is the id of process from which the data is to be received
- **tag** is an identifier for the communication
- **status** is a pointer to a structure that contains information about the communication
 - To ignore use **MPI_STATUS_IGNORE**



Simple point-to-point communication

- This program sends data from process zero to all other processes (a different random number sent to processes):

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);

    int tag_num = 1;

    if (id == 0)
    {
        for (int i = 1; i < p; i++)
        {
            int send_data = rand();
            MPI_Send(&send_data, 1, MPI_INT, i, tag_num, MPI_COMM_WORLD);
            cout << send_data << " sent to processor " << i << endl;
            cout.flush();
        }
    }
    else
    {
        int recv_data;
        MPI_Recv(&recv_data, 1, MPI_INT, 0, tag_num, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        cout << recv_data << " received on processor " << id << endl;
        cout.flush();
    }

    MPI_Finalize();
}
```



Random Number

- `rand` generates an integer random number between 0 and `RAND_MAX`
- `srand` seeds the random number generator
- Both of these functions are found in `<stdlib.h>`
- `srand(time(NULL)+id*10)`
 - `time(NULL)` gives a time as the number of seconds since the beginning of 1970
 - As all the processes will start at the same time, generate different random numbers by adding their id times 10



MPI_Sendrecv

```
int MPI_Sendrecv(  
    void *senddata,  
    int sendcount,  
    MPI_Datatype senddatatype,  
    int destination,  
    int sendtag,  
    void *receivedata,  
    int receivecount,  
    MPI_Datatype recvtype,  
    int source,  
    int receivetag,  
    MPI_Comm communicator,  
    MPI_Status *status)
```

- **Quite** often communications are paired in that a processes is sending and receiving a set of data
- This can be done as two separate commands (and they are always two communications), but they can be set up using a single function **MPI_Sendrecv**
- The parameters for this command are a combination of those for the **MPI_Send** and **MPI_Recv**



MPI Types and their C/C++ equivalents

MPI_SHORT	short int
MPI_INT	int
MPI_LONG	long int
MPI_LONG_LONG	long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	char
MPI_C_BOOL	bool (strictly MPI_CXX_BOOL is for bool, though not all versions of MPI have this type and I know of no implementation for which it is required) or _Bool



MPI return codes

- Most MPI communications give a return value that indicates either success or what went wrong. These are some those codes:

MPI_SUCCESS

- Communication completed successfully

MPI_ERR_COMM

- Invalid communicator

MPI_ERR_COUNT

- Invalid count argument (usually a negative count – a zero, indicating an empty communication, is usually valid)

MPI_ERR_TYPE

- Invalid datatype argument (Either not a valid MPI type or, for types you created yourself, an type that has not been committed – more on this in a later lecture)

MPI_ERR_TAG

- Invalid tag argument

MPI_ERR_RANK

- Invalid source or destination rank. Ranks must be between zero and the size of the communicator minus one



Using a Probe

- A probe allows information about a communication to be read before the communication is completed
 - i.e. before `MPI_Recv` is called in the context of blocking communications
- Two very useful pieces of information can be obtained by doing this
 - Which process has sent the data
 - ...and how much data is being sent
 - You need to be able to assign enough memory to the data pointer that is receiving the data
 - you wish to assign the correct amount of memory for the amount of data being sent you can either send a separate communication saying how much data to expect or you can use a probe to find out



MPI_Probe

- `MPI_Probe` is the function that probes the communication:

```
int MPI_Probe(  
    int source,  
    int tag,  
    MPI_Comm comm,  
    MPI_Status *status)
```

- `source` is the source process
 - `tag` for the communication
 - `comm` is the communicator
 - `status` is a pointer to the structure containing the status information
-
- Having to specify the source and the tag does seem to remove some of the utility of the function, but `MPI_ANY_SOURCE` and/or `MPI_ANY_TAG` can be set



Using the information from MPI_Status

```
typedef struct {  
    int count;  
    int cancelled;  
    int MPI_SOURCE;  
    int MPI_TAG;  
    int MPI_ERROR;  
} MPI_Status;
```

- **count** is the number of received entries
 - Use **MPI_Get_count** in conjunction with the status rather than using this variable directly – avoids potential issues with mismatches in variable type or size
- **cancelled** is true or false depending on whether the corresponding communication was cancelled
- **MPI_SOURCE** is the source process for the communication
- **MPI_TAG** is the tag associated with the communication
- **MPI_ERROR** is an error code (there are a lot of potential errors and you can look them up if you want)
 - Has a value of **MPI_SUCCESS** if there is no error



MPI_Get_count

- `MPI_Get_count` is used in conjunction with the status to get the number of items (not bytes) sent:

```
int MPI_Get_count(  
    const MPI_Status *status,  
    MPI_Datatype datatype,  
    int *count)
```

- `status` is a pointer to the status structure
- `datatype` of the data being sent
- `count` is a pointer to an integer that will store the number of items being sent
- Note that the return value is again an error code and not the number of items (which is stored in the variable pointed to by `count`)



Using MPI_Probe to get the size of the data

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);

    int tag_num = 1;

    if (id == 0)
    {
        for (int i = 1; i < p; i++)
        {
            int num_send = 1 + rand() % 5;
            int *send_data = new int[num_send];
            for (int j = 0; j < num_send; j++) send_data[j] = rand();

            MPI_Send(send_data, num_send, MPI_INT, i, tag_num, MPI_COMM_WORLD);
```

```
        for (int j = 0; j < num_send; j++) cout << send_data[j] << "\t";
        cout << " sent to processor " << i << endl;
        cout.flush();

        delete[] send_data;
    }
}
else
{
    int *recv_data;
    int num_recv;
    MPI_Status status;
    MPI_Probe(0, tag_num, MPI_COMM_WORLD, &status);
    MPI_Get_count(&status, MPI_INT, &num_recv);
    recv_data = new int[num_recv];
    MPI_Recv(recv_data, num_recv, MPI_INT, 0, tag_num, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
    for (int j = 0; j < num_recv; j++) cout << recv_data[j] << "\t";
    cout << " received on processor " << id << endl;
    cout.flush();
    delete[] recv_data;
}
MPI_Finalize();
}
```



What is wrong with blocking communications?

- The main problem with blocking communications is that it is hard to do them efficiently if a lot of communications are required
 - Either you can let each process communicate in turn
 - Lots of idle time waiting for other processes to finish their communications
 - ...or you can try and do clever ordering of the communications so that all processes get to do communications at the same time
 - Easy to get wrong – Will block and crash if, for instance, two process are trying to send data to one another or receive data from one another at the same time
 - Still likely to be inefficient as even efficient ordering will rely on, for instance, an assumption that all communications take about the same time
- Things can be much more efficient if communications don't block one another
 - MPI provides the tools to do this
 - More complex to implement, but potentially much more efficient code
- Note that some MPI implementations of blocking communications don't block when you might think that they should
 - Some implementations of MPI implement some blocking communications using an underlying non-blocking framework
 - Don't rely on this as it is very implementation specific
 - You also might get away with it when sending small amounts of data, but end up blocking with large amounts of data
 - All that a blocking send and receive really guarantees is that the data is safe to use/change after the send or receive exits – Don't expect it to always behave like a barrier, though be aware that it may well do



Non-blocking Communications

- Previously you will have seen how easily communications can block one another
- Non-blocking communications get around this problem
- The basic method in non-blocking communications is as follows
 - Set up all the sends and receives
 - Potentially do some computation while the communication occurs, i.e., occur in the background
 - Wait for communications to complete



An example with non-blocking sends and receives

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);

    int tag_num = 1;

    if (id == 0)
    {
        MPI_Request* request=new MPI_Request[p-1];
        int *send_data = new int[p-1];

        for (int i = 1; i < p; i++)
        {
            send_data[i - 1] = rand();
            MPI_Isend(&send_data[i-1], 1, MPI_INT, i, tag_num, MPI_COMM_WORLD, &request[i-1]);
            cout << send_data[i-1] << " sent to processor " << i << endl;
            cout.flush();
        }
        MPI_Waitall(p - 1, request, MPI_STATUS_IGNORE);

        delete[] send_data;
        delete[] request;
    }
    else
    {
        int recv_data;
        MPI_Request request;
        MPI_Irecv(&recv_data, 1, MPI_INT, 0, tag_num, MPI_COMM_WORLD, &request);

        MPI_Wait(&request, MPI_STATUS_IGNORE);

        cout << recv_data << " received on processor " << id << endl;
        cout.flush();
    }
    MPI_Finalize();
}
```




Non-blocking Communications

- At first glance this might seem to be doing exactly the same thing as the first example in the blocking communication lecture
 - Overall it does achieve the same thing, but some important differences:
 - The `MPI_Isend` and `MPI_Irecv` exit straight away without waiting for the communications to complete
 - The data will be sent in the background. You can carry out computations while the data is being sent:

```
MPI_Irecv(&recv_data, 1, MPI_INT, 0, tag_num, MPI_COMM_WORLD, &request);  
//You can do stuff here that doesn't require the communication to have finished  
MPI_Wait(&request, MPI_STATUS_IGNORE);
```
 - The data will be sent from processor zero to whichever process is first able to receive the data
- Note that I have an array to store the send data. This is so that all values are still available until `MPI_Waitall`
 - Data that is being sent must NOT be changed or deallocated until the communications are actually complete – Guaranteed by `MPI_Waitall`
 - Data that is being received is not available for use until communications are complete



Blocking and Non-blocking communications

- Note that in the previous example I could have used `MPI_Recv` rather than an `MPI_Irecv` and an `MPI_Wait`
 - There is only one communication on the processes other than 0
- `MPI_Recv` can receive data sent by `MPI_Isend` and vice versa
- In this case `MPI_Isend` is the best choice to send the data
 - Communications can occur in the order that the other processes are ready to receive rather than in the order of the process id as would be the case with a loop containing `MPI_Send`
 - The receive in this case makes little difference whether it is blocking or non-blocking
 - It would make a difference if we wanted to do some computation while waiting for the communications to complete



MPI_Isend and MPI_Irecv

```
int MPI_Isend(  
    const void *buf,  
    int count,  
    MPI_Datatype datatype,  
    int dest,  
    int tag,  
    MPI_Comm comm,  
    MPI_Request *request)
```

```
int MPI_Irecv(  
    void *buf,  
    int count,  
    MPI_Datatype datatype,  
    int source,  
    int tag,  
    MPI_Comm comm,  
    MPI_Request * request)
```

- You will notice that `MPI_Isend` and `MPI_Irecv` are very similar to `MPI_Send` and `MPI_Recv`. There are two difference:
 - They both include a `request` variable – more on that later
 - `MPI_Irecv` does not have a status variable – this is because the receive is not blocking and so `MPI_Irecv` actually exits before the communication is necessarily finished and so we do not know the status of the communication at this point



MPI_Wait and MPI_Waitall

- While it is useful to do things while waiting for the communications to finish, at some point you need to be able ensure that a communication has completed so that you can use the data that has been sent
 - This is what `MPI_Wait` and `MPI_Waitall` achieves
 - `MPI_Wait` is used to wait for a single communication to complete
 - `MPI_Waitall` is used to wait for a list of communications to complete
- Note that `MPI_Wait` and `MPI_Waitall` are blocking for the specific communications involved
 - ...or more correctly will continue once the variables used in the communications are safe to change/use
 - For some sends the data might be copied to a buffer and so the code might be able to continue even though the other processor hasn't actually finished receiving the data. This is very similar to the reason why some blocking sends don't actually block when you might expect them to
 - The code will continue once its specific communications are finished, irrespective of communications involving other processes



MPI_Wait and MPI_Waitall

```
int MPI_Wait(  
    MPI_Request *request,  
    MPI_Status *status)
```

```
int MPI_Waitall(int count,  
    MPI_Request *request_list,  
    MPI_Status *status_list)
```

- `request` is a pointer to a single request object
 - This must match a single `MPI_Isend` or `MPI_Irecv`
- `status` is a pointer to an object which will receive the communication status
 - This can be `MPI_STATUS_IGNORE` if you do not need the status information
- `count` is the number of communications that `MPI_Waitall` will be processing
 - These can be a combination of `MPI_Isend` and `MPI_Irecv`s
- `request_list` is an array that contains `count` request objects, one for each communication
- `status_list` is an array of status objects in which the status of each communication will be stored
 - This can again be `MPI_STATUS_IGNORE`



Sending from everyone to everyone

- This is now easy to achieve using non-blocking communications, but this is still less efficient than a collective operation (which we will cover later)
 - Typically do non-blocking communications such as this where processes are communicating with a subset of the other processes and where these subsets overlap
 - Domain decomposition is an example of this type of problem



Sending from everyone to everyone

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);

    int tag_num = 1;

    MPI_Request* request = new MPI_Request[(p - 1)*2];
    double *send_data = new double[p];
    double *recv_data = new double[p];

    int cnt = 0;
```

```
    for (int i=0;i<p;i++)
        if (i != id)
        {
            MPI_Irecv(&recv_data[i], 1, MPI_DOUBLE, i, tag_num, MPI_COMM_WORLD, &request[cnt]);
            cnt++;
        }
        else recv_data[i] = 0;

    for (int i = 0; i<p; i++)
        if (i != id)
        {
            send_data[i] = (double)id / (double)p;
            MPI_Isend(&send_data[i], 1, MPI_DOUBLE, i, tag_num, MPI_COMM_WORLD, &request[cnt]);
            cnt++;
        }
        else send_data[i] = 0;

    MPI_Waitall(cnt, request, MPI_STATUS_IGNORE);

    for (int i = 0; i < p; i++)    cout << "Processor " << id << " recieved " << recv_data[i] << " from processor " << i << endl;

    delete[] request;
    delete[] recv_data;
    delete[] send_data;

    MPI_Finalize();
}
```



A couple of things to notice...

- With the `MPI_Isend` I had to store the values of each of the data points as a separate variable in an array
 - This is because the data can be sent anytime until the `MPI_Waitall` is called
 - This means that the data musn't be changed/overwritten or go out of scope within this interval
 - This is different to `MPI_Send`, where the data to be sent is buffered and/or actually sent by the time `MPI_Send` completes
 - The same is true of `MPI_Irecv`, though this is more obvious as you want to have the data after the `MPI_Wait` / `MPI_Waitall`
- You will notice that I set up the receives before the sends. This is because data can be sent as soon as there is a matching pair of sends and receives
 - Therefore slightly more efficient to have receive ready and waiting for a send
 - Possibly even more efficient to have receives and sends interleaved with an ordering such that some of the communications can get started while others are still being set up
 - This is likely to result in a more complex code structure and may not be worth it for the small efficiency gain
 - Remember that all the non-blocking communications are occurring in the background on a separate communications thread



A side note... Timing your code

- You will often want to know how efficient your code is and so may wish to time it
 - This is a timed version of the previous program
 - Note that there are lots of different potential libraries that would allow timing
 - I use **chrono** to get millisecond precision

```
#include <mpi.h>
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <chrono>

#define DO_TIMING

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL) + id * 10);

    int tag_num = 1;

    MPI_Request* request = new MPI_Request[(p - 1)*2];
    double *send_data = new double[p];
    double *recv_data = new double[p];

#ifdef DO_TIMING
    //The timing starts here - I deliberately exclude the initialisation of MPI, srand and the memory allocation
    //You need to decide what it is that you are actually timing
    auto start = chrono::high_resolution_clock::now();
#endif

    int cnt = 0;
```

```
    for (int i=0; i<p; i++)
        if (i != id)
        {
            MPI_Irecv(&recv_data[i], 1, MPI_DOUBLE, i, tag_num, MPI_COMM_WORLD, &request[cnt]);
            cnt++;
        }
        else recv_data[i] = 0;

    for (int i = 0; i < p; i++)
        if (i != id)
        {
            send_data[i] = (double)id / (double)p;
            MPI_Isend(&send_data[i], 1, MPI_DOUBLE, i, tag_num, MPI_COMM_WORLD, &request[cnt]);
            cnt++;
        }
        else send_data[i] = 0;

    MPI_Waitall(cnt, request, MPI_STATUS_IGNORE);

#ifdef DO_TIMING
    //Note that I exclude couts when timing the code
    //Things like file writes can also be excluded to get a fairer reading
    for (int i = 0; i < p; i++)
        cout << "Processor " << id << " recieved " << recv_data[i] << " from processor " << i << endl;
#endif

#ifdef DO_TIMING
    //Note that this should be done after a block in case process zero finishes quicker than the others
    MPI_Barrier(MPI_COMM_WORLD);
    auto finish = chrono::high_resolution_clock::now();
    if (id == 0)
    {
        std::chrono::duration<double> elapsed = finish - start;
        cout << setprecision(5);
        cout << "The code took " << elapsed.count() << "s to run" << endl;
    }
#endif

    delete[] request;
    delete[] recv_data;
    delete[] send_data;

    MPI_Finalize();
}
```



Checking if communications are open

- In non-blocking communications it is sometimes useful to check if the communications are finished without requiring them to be finished
 - For instance, you could have a loop in which you do some work that does not require the data communication to be finished, with the loop continuing until the communication is finished
 - If you are repeatedly sending more data to a process to do calculations on, you could create the next receive as soon as the previous send has been done and then check if the communication has completed, more data being sent as soon as it completes
 - This is the basis for worksheet 2 workshop exercise 1
- There are two functions for achieving this – `MPI_Test` and `MPI_Testall`



MPI_Test and MPI_Testall

```
int MPI_Test(  
    MPI_Request *request,  
    int *flag,  
    MPI_Status *status)
```

```
int MPI_Testall(int count,  
    MPI_Request *request_list,  
    int *flag,  
    MPI_Status *status_list)
```

- These functions are similar to [MPI_Wait](#) and [MPI_Waitall](#) in terms of the required parameters
- The difference is the [flag](#) parameter, which is a pointer to an integer that will be 1 or 0 (true or false) depending on whether the communications associated with [request](#) or [request_list](#) have completed or not
 - Note that in [MPI_Testall](#) the flag will be true only if all the communications are finished



Example using MPI_Test and MPI_Testall

- In this example we send a lot of data and do things (albeit not very useful things!) while waiting for the communication to finish

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

void Do_Work(void)    //Some (not very useful) work
{
    int sum = 0;
    for (int i = 0; i < 100; i++) sum = sum + 10;
}

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);
```

```
int tag_num = 1, sent_num = 100000, cnt = 0, flag = 0;

if (id == 0)
{
    MPI_Request* request = new MPI_Request[p - 1];
    int **send_data = new int*[p - 1];
    for (int i = 0; i < p - 1; i++)
    {
        send_data[i] = new int[sent_num];
        for (int j = 0; j < sent_num; j++)
            send_data[i][j] = rand();
    }

    for (int i = 1; i < p; i++)
        MPI_Isend(send_data[i-1], sent_num, MPI_INT, i, tag_num, MPI_COMM_WORLD, &request[i - 1]);

    while (MPI_Testall(p - 1, request, &flag, MPI_STATUS_IGNORE) == MPI_SUCCESS && flag == 0)
    {
        Do_Work();
        cnt++;
    }
}
```



Example using MPI_Test and MPI_Testall

```
for (int i = 0; i < p - 1; i++)
    delete[] send_data[i];
delete[] send_data;
delete[] request;
}
else
{
    int *recv_data=new int[sent_num];
    MPI_Request request;
    MPI_Irecv(recv_data, sent_num, MPI_INT, 0, tag_num, MPI_COMM_WORLD, &request);

    while (MPI_Test(&request, &flag, MPI_STATUS_IGNORE) == MPI_SUCCESS && flag == 0)
    {
        Do_Work();
        cnt++;
    }

    delete[] recv_data;
}

cout << "Process " << id << " did " << cnt << " cycles of work while waiting " << endl;
cout.flush();

MPI_Finalize();
}
```

- Note that I don't need to use the `MPI_Wait` or `MPI_Waitall` as I know that all the communications are complete when the while loop exits
- The reason why I have `(MPI_Test(&request, &flag, MPI_STATUS_IGNORE) == MPI_SUCCESS && flag == 0)` as the condition is so that I can both call `MPI_Test` and check the value of the flag within the same condition
 - In C/C++ conditions are evaluated from left to right and therefore the `MPI_Test` is done first
 - The check for `MPI_SUCCESS` is not in expectation of it not being a success, but rather so that the function can be called – If the return is not a success it implies a communication failure and the program is likely to crash anyway



Communications between all processes

- We have shown that non-blocking communications help when sending data between multiple processes
 - Also allows you to do things while waiting
 - Will also continue as soon as a communication is finished – communications don't need to “wait their turn”
- Non-blocking communications are generally better than blocking communications
 - Best alternative when communicating with a number of “neighbouring processes”, but not all the processes
 - E.g. processes connected as graph
 - This is what will be the case in, for instance, domain decomposition – more on this later
- If you want to communicate between all processes at the same time collective communications are generally best



A side note: Using Standard Template Library containers with MPI

- I have thus far been doing examples where memory is allocated using `new` directly
- You can use STL data types, but you need to watch out for a few things:
 - Data must be allocated to the container before using send or receive
 - Data must not move location in memory between the send or receive being setup and the communication being carried out
 - E.g. use `resize` before doing any sends or receives and DON'T use `push_back`
 - `push_back` can cause new data to be allocated and thus cause existing data to move location
 - When transferring more than one item of the same type at the same time you need to use a container that is guaranteed to store the items in contiguous memory
 - E.g. `vector` or `array` are fine as the standard demands contiguous memory be used, `map` or `set` are not fine as the internal memory format is not specified by the standard and all the implementations that I know do not use contiguous memory as that would be inefficient for these container types



The first example using containers

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <vector>
```

```
using namespace std;
```

```
int id, p;
```

```
int main(int argc, char *argv[])
{
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);
```

```
    int tag_num = 1;
```

```
    if (id == 0)
```

```
    {
```

```
        vector<MPI_Request> request;
        vector<int> send_data;
```

```
        request.resize(p-1);
        send_data.resize(p-1);
```

```
        for (int i = 1; i < p; i++)
```

```
        {
```

```
            send_data[i - 1] = rand();
            MPI_Isend(&send_data[i-1], 1, MPI_INT, i, tag_num, MPI_COMM_WORLD, &request[i-1]);
            cout << send_data[i-1] << " sent to processor " << i << endl;
            cout.flush();
```

```
        }
```

```
        MPI_Waitall(p - 1, request.data(), MPI_STATUS_IGNORE);
        //Note the use of request.data() rather than request – I could also use &request[0] instead
```

```
    }
```

```
    else
```

```
    {
```

```
        int recv_data;
        MPI_Request request;
        MPI_Irecv(&recv_data, 1, MPI_INT, 0, tag_num, MPI_COMM_WORLD, &request);
```

```
        MPI_Wait(&request, MPI_STATUS_IGNORE);
```

```
        cout << recv_data << " received on processor " << id << endl;
        cout.flush();
```

```
    }
```

```
    MPI_Finalize();
```

```
}
```



Introduction to Collective Communications

- Thus far all communications we have looked at have been point-to-point
 - Single sending matched to a single receiving process
- Now we will look at collective communications
 - Multiple processes involved in a single communication event
- Same result could be achieved using multiple point to point communications
- ...,but often less efficient
 - In collective communications information can be passed from processor to processor spreading the load
 - Can often achieve a speed scaling of close to $\log(p)$ rather than being linear in p as would be the case when using point to point communications



Broadcast using MPI_Bcast

```
int MPI_Bcast(  
    void *buffer,  
    int count,  
    MPI_Datatype datatype,  
    int root,  
    MPI_Comm comm )
```

- Simplest, but one of the most useful collective operations
- Sends the same data from one process to all other processes
 - Needs to be called at the same time by all processes
- `MPI_Bcast` is blocking
- Function definition looks like an `MPI_Send`, but the source, rather than the destination, is specified by `root`
 - The destination is every other process
- On root process `buffer` is a pointer to the data to be sent
- On other processes `buffer` is a pointer to where the sent data is to be stored
 - After the operation `buffer` will contain the same data on all processes



Broadcast using MPI_Bcast

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL)+id*10);

    int num_send = 1;
    int *data = new int[num_send];

    if (id == 0)
        for (int i = 0; i < num_send; i++)
            data[i] = rand();

    MPI_Bcast(data, num_send, MPI_INT, 0, MPI_COMM_WORLD);

    if (id == 0) cout << "Process 0 sent this data: ";
    else cout << "Process " << id << " received this data: ";

    for (int i = 0; i < num_send; i++)
        cout << "\t" << data[i];
    cout << endl;
    cout.flush();

    delete[] data;

    MPI_Finalize();
}
```



Gather and Scatter Communications

- Broadcast sends the same data to all the processes from a single process
 - Note that, in the previous example, `MPI_Bcast` is called on all processes at the same time. This is true of all collective operations.
- Sometimes you wish to send different data to each of the processes, but originating at a single process
 - This is known as a Scatter operation
- The reverse of a Scatter operation is a Gather operation
 - Gather involves sending data from all the processes and collecting it on a single process



MPI_Scatter

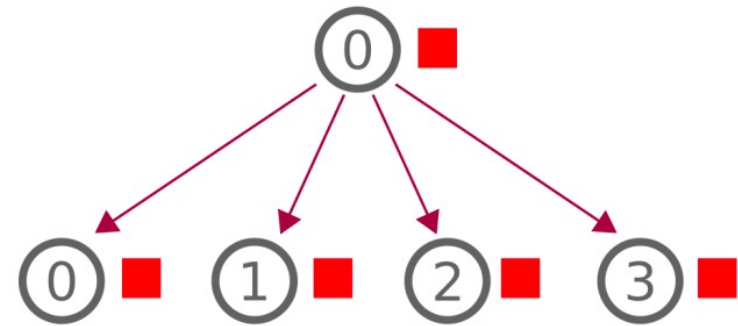
```
int MPI_Scatter(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    int recv_count,  
    MPI_Datatype recv_datatype,  
    int root,  
    MPI_Comm communicator)
```

- `send_data` is a pointer to the location of the source data
 - Note that this is only important on the `root` and can be a `nullptr` on other processes
- `send_count` is the number of items to be sent to each process, not the total amount data
 - E.g. if you are sending 10 integers to each process and there are 12 processes involved then `send_data` will need to contain space for 120 integers, but `send_count` will be 10
- `recv_data` is a pointer to the location where the data is to be stored
 - Cannot be `nullptr` on `root` as it will also receive its share of the data
- `recv_count` is the number of items expected
 - Generally this would be the same as the `send_count`, but could be different if, for instance, you sent the data as bytes and received then as integers

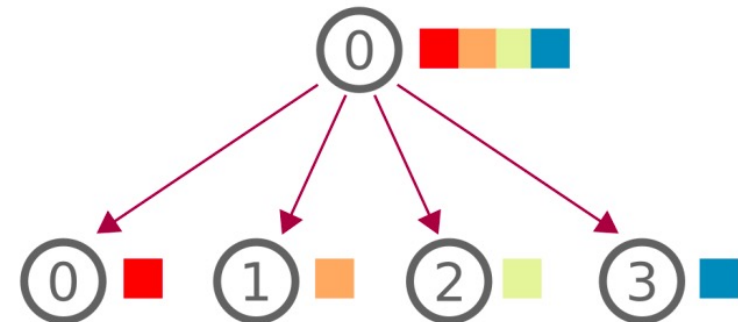
The structure of send_data

- With `MPI_Scatter`, the data in `send_data` is sent to processes in the order of the process id
- If `send_count` is greater than one then the first `send_count` items are sent to process zero, the second `send_count` items to process one etc.
- The size of `send_data` should therefore be `send_count` times the total number of processes
 - If you wish to scatter an amount of data not exactly divisible by the number of processes you may need to pad the data
- Note that the data for the root is also copied into `recv_data` on the root

MPI_Bcast



MPI_Scatter





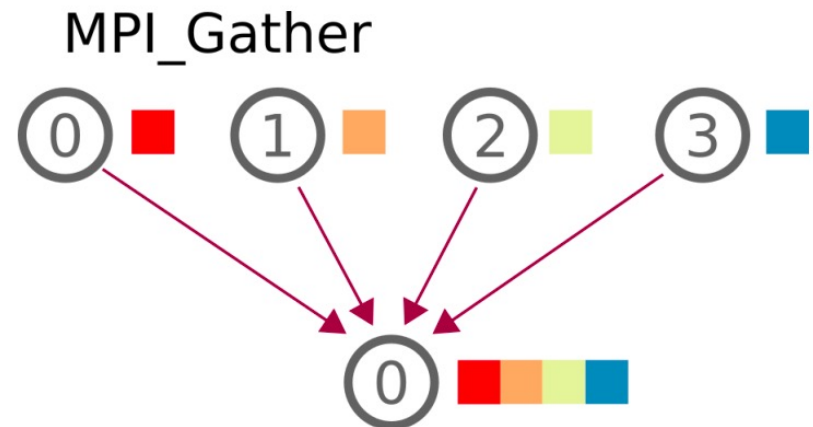
MPI_Gather

```
int MPI_Gather(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    int recv_count,  
    MPI_Datatype recv_datatype,  
    int root,  
    MPI_Comm communicator)
```

- The parameters required for `MPI_Gather` are identical to that for `MPI_Scatter`
- The big difference is that now the `send_data` has only got `send_count` items, while `recv_data` will need space for `recv_count` times the number of processes of data
 - This time it is the `recv_data` that can be `NULL` (or `nullptr`) on processes other than the root

The structure of recv_data

- As MPI_Gather is essentially the reverse operation of MPI_Scatter, the structure of the data in `recv_data` will be similar to that in `send_data` from MPI_Scatter
 - `recv_data` will have the data sent from process zero stored in the first `recv_count` elements, the data for process one stored in the next `recv_count` elements etc.





An example using MPI_Scatter and MPI_Gather

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

double calc_ave(double *list, int num)
{
    double total = 0.0;
    for (int i = 0; i < num; i++)
        total += list[i];
    return total / num;
}

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL) + id * 10);

    double *send_data1 = nullptr, send_data2;
    double *recv_data1 = nullptr, *recv_data2 = nullptr;
    int num_sendrecv = 20;
```

```
    if (id == 0)
    {
        send_data1 = new double[num_sendrecv*p];
        for (int i = 0; i < num_sendrecv*p; i++)
            send_data1[i] = ((double)rand() / (double)RAND_MAX)*100.0;
    }

    recv_data1 = new double[num_sendrecv];

    MPI_Scatter(send_data1, num_sendrecv, MPI_DOUBLE, recv_data1, num_sendrecv, MPI_DOUBLE, 0,
        MPI_COMM_WORLD);

    send_data2 = calc_ave(recv_data1, num_sendrecv);

    delete[] recv_data1;    //Can be called on all processes as it is nullptr where not used
    delete[] send_data1;

    if (id == 0)
        recv_data2 = new double[p];

    MPI_Gather(&send_data2, 1, MPI_DOUBLE, recv_data2, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    if (id == 0)
    {
        cout << "The following averages were calculated for each set of data:" << endl;
        for (int i = 0; i < p; i++)
            cout << "\t" << i << " " << recv_data2[i] << endl;
    }
    delete[] recv_data2;
    MPI_Finalize();
}
```



MPI_Gatherv and MPI_Scatterv

- In the standard `MPI_Gather` and `MPI_Scatter` the same amount of data is being received or sent by each process.
- Sometimes it is useful to send different amounts of data to different processes
- This can be achieved using the `MPI_Gatherv` and `MPI_Scatterv`
- The difference is that there needs to be an array specifying the amount of data to be received from or sent to each of the other processes on the originating process
- It also allows data to be sent with different displacements in the originating or final array



MPI_Gatherv

```
int MPI_Gatherv(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    const int *recv_counts,  
    const int *displacements,  
    MPI_Datatype recv_datatype,  
    int root,  
    MPI_Comm communicator)
```

- Most parameters are the same as for `MPI_Gather`
- The big difference is that there are two new parameters:
- `recv_counts` is an array of integers giving the amount of data to be received from each process. This should correspond to the `send_count` on each of the processes
- `displacements` is the displacement from `recv_data` where the received data from each process is to be stored. If this is the progressive sum of `recv_counts` then the data will be tightly packed



MPI_Scatterv

- Most parameters are the same as for MPI_Scatter
- The big difference is that there are two new parameters: your program
 - `send_counts` is an array of integers giving the amount of data to be sent to each process. This should correspond to the `recv_count` on each of the processes
 - `displacements` is the displacement from `send_data` where the sent data for each process starts. If this is the progressive sum of `send_counts` then there will be no gaps between the data to be sent



An example using MPI_Scatter and MPI_Scatterv

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

const int max_send = 5;

int id, p;
int* send_data = nullptr, *recv_data, *num_send = nullptr; *displacements = nullptr;
int num_receive, num_send_tot;

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL) + id * 10);

    int tag_num = 1;

    if (id == 0)
    {
        num_send = new int[p];
        displacements = new int[p];
        num_send_tot = 0;
        displacements[0] = 0;
        for (int i = 0; i < p; i++)
        {
            num_send[i] = 1 + rand() % max_send;
            num_send_tot += num_send[i];
            if (i > 0) displacements[i] = displacements[i - 1] + num_send[i - 1];

            cout << num_send[i] << " items being sent to process " << i << endl;
        }
    }
```

```
    send_data = new int[num_send_tot];
    cout << "Array being sent: ";
    for (int i = 0; i < num_send_tot; i++)
    {
        send_data[i] = rand() % 1000;
        cout << "\t" << send_data[i];
    }
    cout << endl;
    cout.flush();
}

MPI_Scatter(num_send, 1, MPI_INT, &num_receive, 1, MPI_INT, 0, MPI_COMM_WORLD);

cout << "Process " << id << " expects to receive " << num_receive << " items" << endl;
recv_data = new int[num_receive];

MPI_Scatterv(send_data, num_send, displacements, MPI_INT, recv_data, num_receive, MPI_INT, 0,
    MPI_COMM_WORLD);

cout << "Process " << id << " received: ";
for (int i = 0; i < num_receive; i++)
    cout << "\t" << recv_data[i];
cout << endl;
cout.flush();

delete[] recv_data;
delete[] num_send;
delete[] displacements;
delete[] send_data;

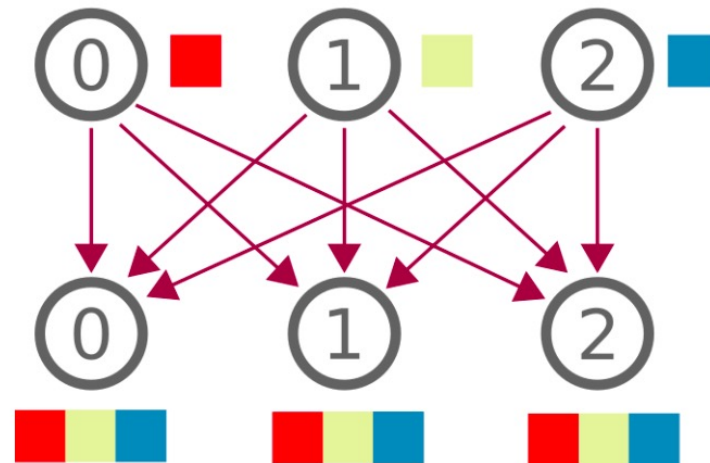
MPI_Finalize();
```

MPI_Allgather

```
int MPI_Allgather(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    int recv_count,  
    MPI_Datatype recv_datatype,  
    MPI_Comm communicator)
```

- `MPI_Allgather` is very closely related to `MPI_Gather`
 - The difference is that the `recv_data` ends up on all processes
 - A `root` is therefore not required

MPI_Allgather





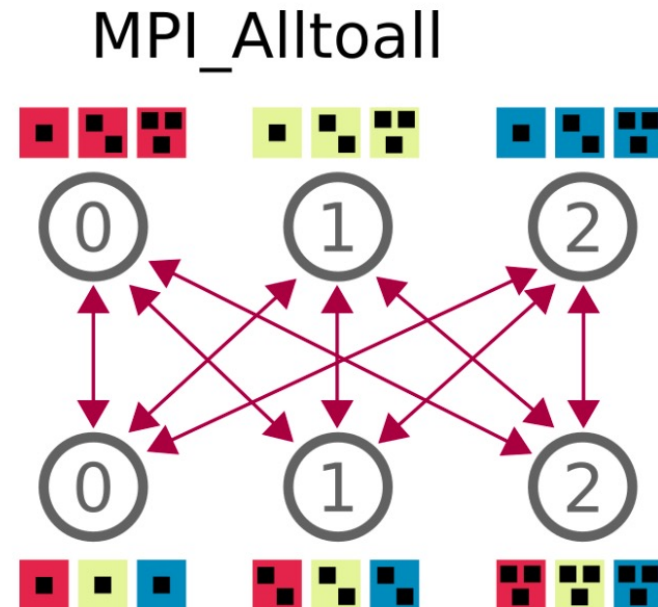
MPI_Alltoall

```
int MPI_Alltoall(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    int recv_count,  
    MPI_Datatype recv_datatype,  
    MPI_Comm communicator)
```

- The parameters in `MPI_Alltoall` are the same as those for `MPI_Allgather`, but what it does is quite different

MPI_Alltoall

- Unlike in `MPI_Allgather`, where each process receives the same data, in `MPI_Alltoall` each process sends a different piece of data to each other process
 - The total size of both `send_data` and `recv_data` must be equal to the count times the number of processes
 - Achieves the same thing as the previous lecture's everyone communicating with everyone example, but is more efficient





MPI_Reduce

- `MPI_Reduce` is a very useful collective operation which shares some similarity to `MPI_Gather` in that data is sent from all the processes and collated on `root`
- Its big difference is that the data is not stored separately, but combined using an operation `op`
 - This is also why there is a single `count` and `datatype` as the operation must involve a single type
 - The operation is applied separately to each of the items in the data
- `MPI_Allreduce` is similar to `MPI_Reduce` except that the data is received on all processes
 - A `root` is therefore not required

```
int MPI_Reduce(  
    void* send_data,  
    void* recv_data,  
    int count,  
    MPI_Datatype datatype,  
    MPI_Op op,  
    int root,  
    MPI_Comm communicator)
```




Reduce operations in MPI (MPI_Op op)

- There are a large number of different reduce operations that are available:

MPI_MAX: Stores the maximum value

MPI_MIN: Stores the minimum value

MPI_SUM: Stores the sum of the values

MPI_PROD: Stores the product of the values

MPI_LAND: Carries out a logical “and” on all the values

MPI_BAND: Carries out a bit-wise “and” on all the values

MPI_LOR: Carries out a logical “or” on all the values

MPI_BOR: Carries out a bit-wise “or” on all the values

MPI_LXOR: Carries out a logical “xor” on all the values

MPI_BXOR: Carries out a bit-wise “xor” on all the values

MPI_MAXLOC: Gives a maximum value and its location (i.e. it returns a pair of values)

MPI_MINLOC: Same as **MPI_MAXLOC**, but based on a minimum value



Example of MPI_Reduce

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

double calc_ave(double *list, int num)
{
    double total = 0.0;
    for (int i = 0; i < num; i++)
        total += list[i];
    return total / num;
}

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL) + id * 10);

    double *send_data1 = nullptr, send_data2;
    double *recv_data1 = nullptr, recv_data2;
    int num_sendrecv = 20;

    if (id == 0)
    {
        send_data1 = new double[num_sendrecv*p];
        for (int i = 0; i < num_sendrecv*p; i++)
            send_data1[i] = ((double)rand() / (double)RAND_MAX)*100.0;
    }

    recv_data1 = new double[num_sendrecv];

    MPI_Scatter(send_data1, num_sendrecv, MPI_DOUBLE, recv_data1, num_sendrecv, MPI_DOUBLE, 0,
        MPI_COMM_WORLD);

    send_data2 = calc_ave(recv_data1, num_sendrecv);

    delete[] recv_data1;    //Can be called on all processes as it is NULL where not used
    delete[] send_data1;

    MPI_Reduce(&send_data2, &recv_data2, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (id == 0)
        cout << "The average of all the data is " << recv_data2 / p << endl;

    MPI_Finalize();
}
```



Non-blocking collectives

- You could use the non-blocking version of the collective operation, which allows you to do things like continuing working while waiting for them to complete
 - We will not specifically look at non-blocking collectives here, but their difference to the blocking version is similar to the difference between, for instance, `MPI_Send` and `MPI_Isend`
 - They also require a request variable and are guaranteed to have completed once `MPI_Wait` has been called
 - They can also be checked for completion using `MPI_Test`
- The non-blocking equivalents all have an `I` in their name. E.g.
 - Non-blocking `MPI_Bcast` is `MPI_Ibcast`
 - Non-blocking `MPI_Gather` is `MPI_Igather` etc.
- The parameters they take in and their usage are identical to their non-blocking equivalents except for the extra `MPI_Request*` variable



Non-blocking collectives example

```
#include <mpi.h>
#include <iostream>
#include <cstdlib>
#include <time.h>

using namespace std;

int id, p;

double calc_ave(double *list, int num)
{
    double total = 0.0;
    for (int i = 0; i < num; i++)
        total += list[i];
    return total / num;
}

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);
    srand(time(NULL) + id * 10);

    int tag_num = 1;
    double *send_data1 = nullptr, send_data2;
    double *recv_data1 = nullptr, recv_data2;
    int num_sendrecv = 20;

    if (id == 0)
    {
        send_data1 = new double[num_sendrecv*p];
        for (int i = 0; i < num_sendrecv*p; i++)
            send_data1[i] = ((double)rand() / (double)RAND_MAX)*100.0;
    }

    recv_data1 = new double[num_sendrecv];

    MPI_Request request;

    MPI_Isscatter(send_data1, num_sendrecv, MPI_DOUBLE, recv_data1, num_sendrecv, MPI_DOUBLE, 0, MPI_COMM_WORLD, &request);

    //Do other work here

    MPI_Wait(&request, MPI_STATUS_IGNORE);

    send_data2 = calc_ave(recv_data1, num_sendrecv);

    delete[] recv_data1;    //Can be called on all processes as it is nullptr where not used
    delete[] send_data1;

    MPI_Ireduce(&send_data2, &recv_data2, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD, &request);

    //Do other work here

    MPI_Wait(&request, MPI_STATUS_IGNORE);

    if (id == 0)
        cout << "The average of all the data is " << recv_data2 / p << endl;

    MPI_Finalize();
}
```



Why create your own MPI datatype?

- When using MPI it is best to do as few communications as possible
 - Rather send all required data in a single large communication than in a number of smaller communications
- If the data is continuous and all of the same type (e.g. an array) this is easy to achieve
- Often you want to send discontinuous information and/or variables of a mixture of types (e.g. only some of the member variables in an object)
- We can achieve this by making our own MPI variable types to encompass a number of simpler MPI types (or even other types that we have created ourselves)



Demonstrate with an example

- Creating an MPI variable to send some of the member variables within an object

```
#include <mpi.h>
#include <iostream>
#include <locale>

using namespace std;

int id, p;

class my_class
{
public:
    int I1, I2;
    int var_not_to_send;
    double D1;
    /*Note that I am using a C string rather than, e.g.std::string
    This is because the data in std::string is dynamically allocated and
    does not have a fixed relationship to the rest of the data in the class*/
    char S1[50];

    static void buildMPIType();
    static MPI_Datatype MPI_type;
};

MPI_Datatype my_class::MPI_type;
```

```
void my_class::buildMPIType()
{
    int block_lengths[4];
    MPI_Aint displacements[4];
    MPI_Aint addresses[4], add_start;
    MPI_Datatype typelist[4];

    my_class temp;

    typelist[0] = MPI_INT;
    block_lengths[0] = 1;
    MPI_Get_address(&temp.I1, &addresses[0]);

    typelist[1] = MPI_INT;
    block_lengths[1] = 1;
    MPI_Get_address(&temp.I2, &addresses[1]);

    typelist[2] = MPI_DOUBLE;
    block_lengths[2] = 1;
    MPI_Get_address(&temp.D1, &addresses[2]);

    typelist[3] = MPI_CHAR;
    block_lengths[3] = 50;
    MPI_Get_address(&temp.S1, &addresses[3]);

    MPI_Get_address(&temp, &add_start);
    for (int i = 0; i < 4; i++) displacements[i] = addresses[i] - add_start;

    MPI_Type_create_struct(4, block_lengths, displacements, typelist, &MPI_type);
    MPI_Type_create_resized(MPI_type, 0, sizeof(my_class), &MPI_type);
    MPI_Type_commit(&MPI_type);
}
```



Continuing the example

```
int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    my_class::buildMPIType();

    my_class data;

    if (id == 0)
    {
        data.I1 = 6;
        data.I2 = 3.0;
        data.D1 = 10.0;
        data.var_not_to_send = 25;
        string str_temp = "My test string";
        //Copy the data from the std:: string into the C string - safer than using strcpy
        copy(str_temp.begin(), str_temp.end(), data.S1);
    }

    MPI_Bcast(&data, 1, my_class::MPI_type, 0, MPI_COMM_WORLD);

    cout << "On process " << id << " I1=" << data.I1 << " I2=" << data.I2 << " D1=" << data.D1 << " S1=" << data.S1 << ". The unsent variable is " << data.var_not_to_send << endl;
    MPI_Type_free(&my_class::MPI_type);
    MPI_Finalize();
}
```




Writing data from multiple nodes

- Data can be transferred back to a single node and written as a single file
 - Fine for smaller amounts of output data
 - Can become very expensive in communication terms if there is a lot of output data and/or data needs to be frequently outputted (e.g. outputting lots of data at multiple timesteps)
 - We will look at the MPI implementation of this using `MPI_file_write` (and its variants) to handle multiple writes to the same file from different processes
- One alternative is to have each process write its own data to file
 - Still need to still be transferred to a single destination, but often to a file server rather than one of the other nodes (which would still need to write it somewhere)
 - Need to carry out post-processing to combine the data from the multiple files
 - Can result in a very large number of small files, which might make you unpopular with the systems administrator!



Numbered File Names

- File names need to be numbered according to the processor that created it and (potentially) the output/iteration/timestep number
 - You don't want to overwrite data from other processes and trying to have different processes write to the same file can result in either errors or serious blocking issues
- Using a string to create the file name and open the file (where `num` is an output step number and `id` is the rank of the process):

```
string fname;
```

```
fname = "output" + to_string(num) + "_" + to_string(id) + ".dat";
```

```
f1.open(fname.c_str(), ios_base::out);
```



Writing files using MPI

- MPI includes its own functionality for reading and writing files
 - Quite a broad topic that we will only touch the surface of
- One useful feature is the ability to write data from multiple processes to a single file either in a ordered or unordered format
 - Most, but not all file systems will support this
 - Will be slower than writing data to individual files on the local drive as it requires network communications



Example writing data to a file

- In this example each file will write a string to a shared file
 - Note that this is just a code snippet

```
MPI_File mpi_file;
string fname("test.txt");

if (MPI_File_open(MPI_COMM_WORLD, fname.c_str(), MPI_MODE_CREATE | MPI_MODE_WRONLY, MPI_INFO_NULL, &mpi_file) != MPI_SUCCESS)
{
    cout << "Error opening file!" << endl;
    exit(0);
}

string string_out;

string_out = string("This data has been written by ") + to_string(id) + string("\n");

MPI_File_write_ordered(mpi_file, string_out.c_str(), (int)string_out.length(), MPI_CHAR, MPI_STATUS_IGNORE);

MPI_File_close(&mpi_file);
```



Opening MPI Files

- In MPI files are handled in a similar way to how they are done in C
 - File handles are used instead of the more C++ way of using streams

```
MPI_File mpi_file;
```

- **MPI_File** is the file handle variable type that is used to address specific files
- The file to be opened, e.g.:

```
MPI_File_open(MPI_COMM_WORLD, fname.c_str(), MPI_MODE_CREATE | MPI_MODE_WRONLY,  
MPI_INFO_NULL, &mpi_file);
```

- This specific example will create a new file for writing to only



MPI_File_open

```
int MPI_File_open(  
    MPI_Comm comm,  
    char *filename,  
    int mode,  
    MPI_Info info,  
    MPI_File *file_handle)
```

- **filename** is the name of the file to be opened
 - This can include paths etc.
- **comm** is the communicator
 - We can stick with **MPI_COMM_WORLD**
- **mode** is the file access mode to use
 - Can be combined using a binary or (|)
- **info** stores information about the file that is being opened
 - Can be ignored using **MPI_INFO_NULL**
- **file_handle** is the pointer to the file handle that is used for the subsequent file writing
- The function will return a status
 - I recommend checking this status - **MPI_SUCCESS** means that the file has been successfully opened



File modes

<code>MPI_MODE_RDONLY</code>	Read only
<code>MPI_MODE_RDWR</code>	Reading and writing
<code>MPI_MODE_WRONLY</code>	Write only
<code>MPI_MODE_CREATE</code>	Create the file if it does not exist
<code>MPI_MODE_EXCL</code>	Error if creating file that already exists
<code>MPI_MODE_DELETE_ON_CLOSE</code>	Delete file on close
<code>MPI_MODE_UNIQUE_OPEN</code>	File will not be opened elsewhere
<code>MPI_MODE_SEQUENTIAL</code>	File will only be accessed sequentially Can't move backwards and forwards in the file
<code>MPI_MODE_APPEND</code>	Add data to end of an existing file



MPI_File_write functions

- There are a number of different types of MPI file write (and equivalent read) functions that can be used
 - **MPI_File_write** – Writes to the file, but each process must set the location for writing
 - By default the file pointer will point at the beginning of the file
 - By default each processor will therefore tend to write over the data of other processors
 - Each processor needs to move its file pointer to an appropriate location to stop this happening
 - Used in combination with **MPI_File_seek** to move the file pointer to the desired location
 - Note that this writing need not be done simultaneously on each processor
 - **MPI_File_write_at** – Write data to a specified location in the file
 - Roughly equivalent to using **MPI_File_seek** followed by **MPI_File_write**, but slightly safer
 - Note that this writing need not be done simultaneously on each processor
 - **MPI_File_write_ordered** – The data written is ordered by the processor order
 - Probably the easiest way of writing data
 - ...,but requires simultaneous writing and can therefore be less efficient as every process needs to wait for the slowest process before writing can occur
 - This is writing to a shared file pointer and requires simultaneous writes (or reads) from all processes
 - This involves a blocking collective communication



MPI求 π 值

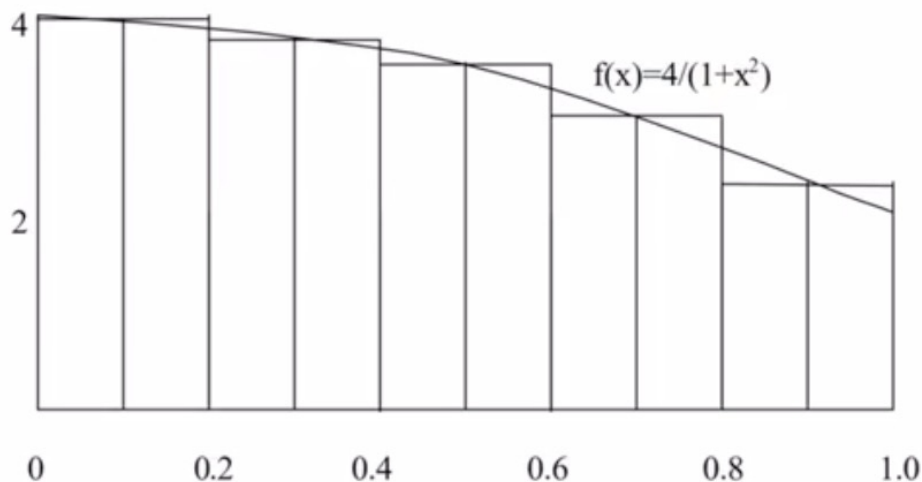
积分公式

$$\int_0^1 \frac{1}{1+x^2} dx = \arctan(x) \Big|_0^1 = \arctan(1) - \arctan(0) = \arctan(1) = \frac{\pi}{4}$$

令函数 $f(x) = 4/(1+x^2)$ 则有

$$\int_0^1 f(x) dx = \pi$$

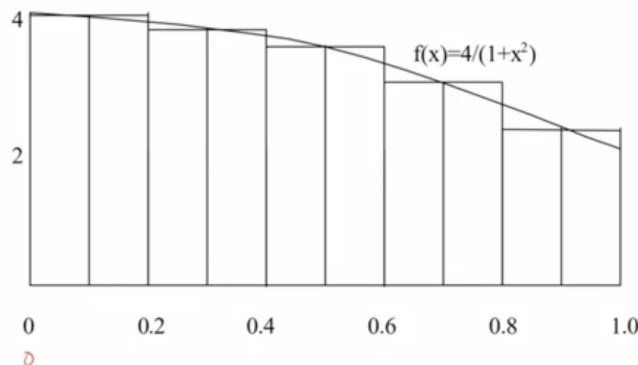
$f(x)$ 图像为



提问:如何利用MPI实现? 涉及哪些算子?



MPI求 π 值



求 π 近似值方法的示意图

计算 $f(x)$ 图象上面从0到1之间的面积即为 π 值, 矩形的高度取函数在矩形中间点的取值, 当用更多的矩形来划分时, 该近似值就越接近于真实的 π 值, 设将0到1的区间划分为 N 个矩形 则近似公式为

$$\pi \approx \sum_{i=1}^N f\left(\frac{2 \times i - 1}{2 \times N}\right) \times \frac{1}{N} = \frac{1}{N} \times \sum_{i=1}^N f\left(\frac{i - 0.5}{N}\right)$$

谢谢！



中國人民大學
RENMIN UNIVERSITY OF CHINA